Typically, for better results, you want to run the application for some time and perform several interactions so that you can accumulate instances of functions that pose heavy memory usage. This approach allows you to observe how the application behaves under typical usage patterns, helping to identify specific areas where memory consumption spikes.

PERFORMANCE.NOW()

The `Performance.now()` API provides high-resolution timestamps that let you measure the time taken by specific operations in your application. This precision is useful for benchmarking performance-critical sections of your code. In the example project, we have an example profiler that uses this function to capture some important performance metrics like duration of calls, streaming metrics, and average latency. Here is an example usage:

```
import { Profiler } from '@/lib/profiler';
   const profiler = new AIPerformanceProfiler();
   profiler.startOperation('initialAIRequest');
   // Call API
   const finalMetrics = profiler.endOperation('completeAIRequest');
   console.log('Performance Metrics:', {
       totalDuration: finalMetrics.duration,
       apiLatency: finalMetrics.duration,
       streamingMetrics: finalMetrics.streamMetrics,
       averageChunkLatency: finalMetrics.streamMetrics.avgChunkLatency
   });
```

The main idea here is to capture performance metrics of costly operations to help optimize the entire pipeline from the initial request to final token delivery. Once you measure important metrics, you can set up a tracker that captures improvements or degradations in performance over time. This allows for continuous monitoring and evaluation of your application's efficiency and an understanding of where optimizations are needed.

8.2 Vercel AI SDK troubleshooting

Many potential problems with the Vercel AI SDK arise because the library relies on external APIs, which can have differences between them. Additionally, major updates to this SDK introduce deprecations and changes to the recommended classes or functions. For example, during the authoring of this book, there has been two major version changes in this SDK, which required me to review existing code to verify its functionality. When developing software that depends on external tools and SDKs, you will need to adopt a similar approach to ensure secure and up-to-date dependencies.

Despite these challenges, we can still follow a methodical approach to identifying and resolving problems related to this library. Figure 8.5 shows some common troubleshooting workflows that can guide you through the process.